

Gradients and Chain Rule:

1. Gradient ∇L :

$$L = f(w_1, w_2, \dots, w_n)$$

$$\frac{\partial L}{\partial w_1} \quad \frac{\partial L}{\partial w_2}$$

$$\nabla L = \left[\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \dots, \frac{\partial L}{\partial w_n} \right]$$

$\nabla L \rightarrow$ List of Partial Derivatives - one per weight.

- gradient points to the direction of increasing loss
- Our goal: reduce loss $\rightarrow$ step in the opposite direction of the gradient

Example:

$$L = w^2 + 3w + 5 \Rightarrow \text{slope} = \frac{\partial L}{\partial w} = 2w + 3$$

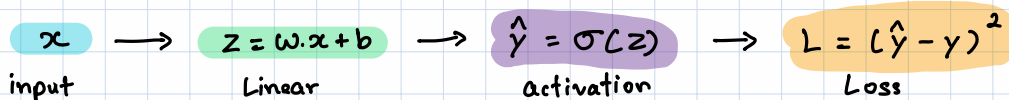
$$\text{Assume } w = 2; \text{ slope (or) } \frac{\partial L}{\partial w} = 2(2) + 3 = +7$$

Inference: slope is +ve ; loss goes up when weight goes up.
So reduce weight "w" to reduce loss "L".
(Vice-versa for -ve slope)

2. Chain Rule:

Purpose : Loss "L" doesn't directly touch weight "w".
it's 3 stages away So, use Chain rule to bridge them.

Functions of a Single Neuron:



$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z} \times \frac{\partial z}{\partial w}$$

$\rightarrow$ This is chain rule
 $\rightarrow$ one function depends on another function

Backward Propagation - 3 pieces:

$$\textcircled{1} \frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y) \quad \textcircled{2} \frac{\partial \hat{y}}{\partial z} = \sigma'(z) \quad \textcircled{3} \frac{\partial z}{\partial w} = x$$

(Sigmoid derivative)

$$\frac{\partial L}{\partial w} = 2(\hat{y} - y) \cdot \sigma'(z) \cdot x$$

→ do this parallelly for all weights



Important

Weight update Rule:

$$w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$$

↓
old weight

↓
new weight

α → Learning rate

$\frac{\partial L}{\partial w}$ → gradient

"-ve" → to go in the opposite dir. of gradient

Backpropagation = Apply chain rule backward through every layer